

Sistemas Distribuidos y Paralelos

Ingeniería en Computación



Paradigmas de programación paralela

Universidad Nacional de La Plata



Facultad de Informática





1

Paradigmas de programación paralela

2

Paralelismo de datos SIMD/SPMD

3

Divide y vencerás

4

Pipelines

5

Algoritmos sistólicos (Heartbeat)

6

Master-Worker

7

Task Pool o Bag of Tasks

8

Paradigmas híbridos

A dark, industrial-style workspace. In the foreground, a desk holds an open book with dense text, a pen, and some papers. In the background, a computer monitor displays data, and the room is filled with various electronic equipment and glowing lights, creating a high-tech, somewhat mysterious atmosphere.

1

Paradigmas de programación paralela

Paradigmas de programación paralela

4

- Programas paralelos nuevos no siempre requieren un diseño completo:
 - Algoritmos paralelos similares, siguen ciertos patrones de distribución del trabajo, comunicación y sincronización.
- Esos patrones se conocen como paradigmas de programación paralela:

Paradigma (o patrón) de programación paralela: Clase de algoritmos que resuelven distintos problemas pero comparten una misma estructura en la forma de distribuir el trabajo, comunicarse y sincronizarse.

- Los paradigmas más comunes:

Paralelismo de datos - SIMD/SPMD

Divide y vencerás

Pipelines

Algoritmos sistólicos

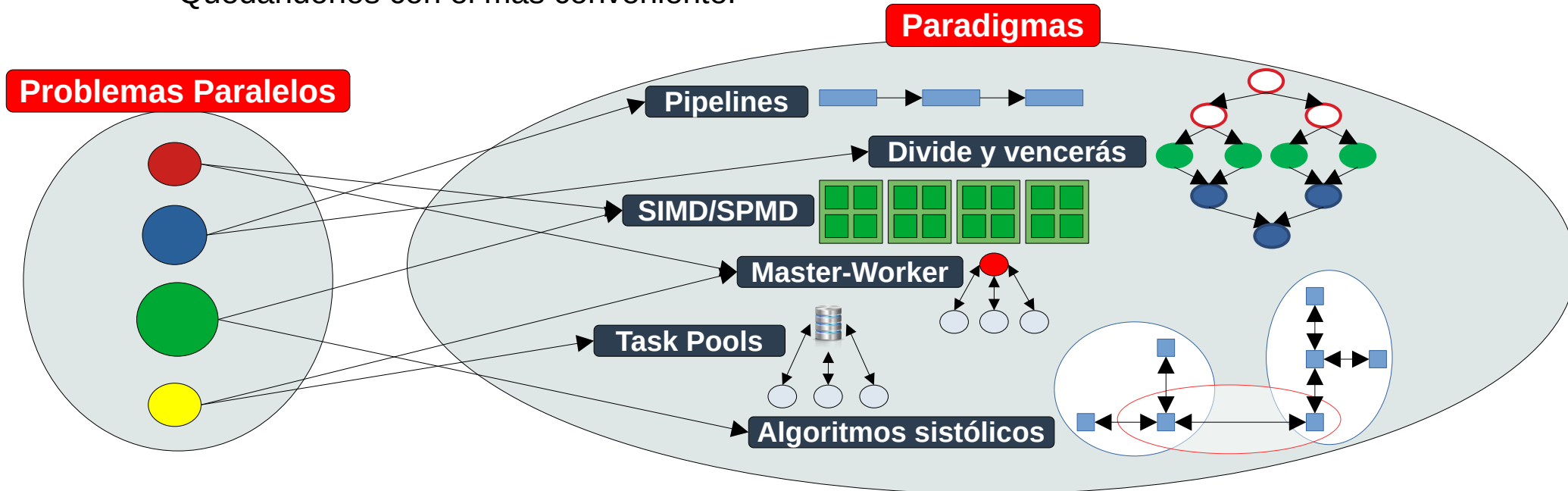
Master-Worker

Task Pools

Paradigmas de programación paralela

5

- Para cada paradigma puede escribirse un esqueleto algorítmico que define la estructura de control común.
- Podemos clasificar tipos de problemas paralelos en uno o mas de estos paradigmas.
 - Quedándonos con el mas conveniente.





2

Paralelismo de datos SIMD/SPMD

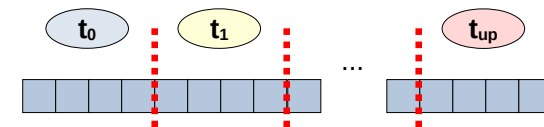
Paralelismo de datos - SIMD/SPMD

7

- Se deriva de la idea de diseño de paralelismo de datos o de dominio.
- Paradigma más común y más simple:
 - **Tareas se mapean estática** o semiestáticamente a unidades de procesamiento.
 - Hilos o procesos realizan **operaciones similares o idénticas** sobre diferentes datos.
 - Algoritmos fáciles de programar y **altamente escalables**.
 - Trabajo en fases con interacciones para sincronizar u obtener datos nuevos.

Aplicación:

- Estructuras regulares y estáticas
- Poca dependencia de datos o ninguna (Embarrassing Parallel Problem)
- Leve dependencia de datos sobre problemas de frontera (Ej: imágenes)
- Simulación



Paralelismo de datos - SIMD/SPMD

8

- Dos modelos:
 - **SIMD (Single Instruction Multiple Data):** hilos o procesos ejecutan sincrónicamente la misma instrucción sobre distintos datos.
 - Cada unidad de procesamiento realiza exactamente la misma operación, pero sobre un elemento diferente del conjunto de datos.
 - GPUs y unidades vectoriales (MMX, SSE y AVX).
 - **SPMD (Single Program Multiple Data):** Cada hilo o proceso ejecuta el mismo programa, asincrónicamente, a distintas velocidades, sobre diferentes datos.
 - No hay obligación de que las instrucciones se ejecuten al mismo tiempo.



3

Divide y vencerás

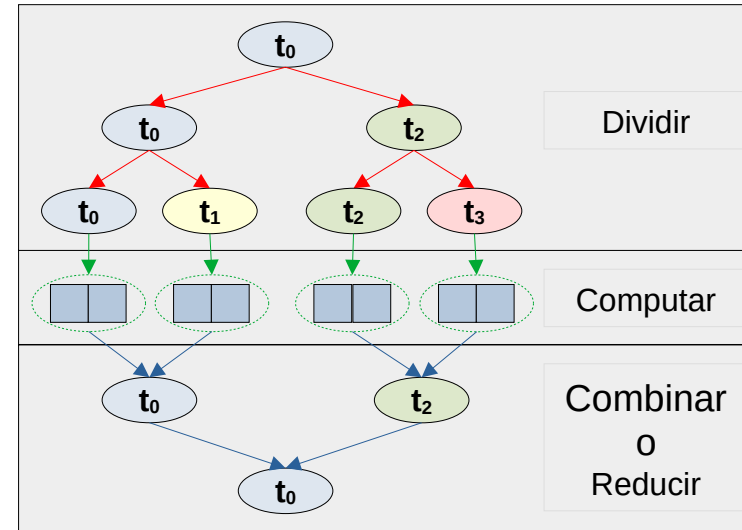
Divide y vencerás

10

- Se deriva de la descomposición recursiva.
- Dividir un problema en subproblemas, que son el mismo problema, de menor tamaño, resolverlos de manera independiente y combinar o reducir los resultados.
- Tres etapas:
 - Dividir
 - Computar
 - Combinar/Reducir

Aplicación:

- Ordenación
- Búsquedas
- Map-Reduce



Divide y vencerás

11

- El paralelismo óptimo depende de los recursos disponibles:
 - No siempre es posible generar tantos hilos o procesos como subproblemas independientes.
 - Se crean hilos o procesos dinámicamente según los recursos.

¿Podemos lograr el **máximo paralelismo** con "*mil millones*" de cálculos independientes?

¿"*mil millones*" de hilos o procesos?

¿"*mil millones*" de unidades de procesamiento?

Simultaneos!!!

No siempre se tienen los recursos para lograr un paralelismo óptimo





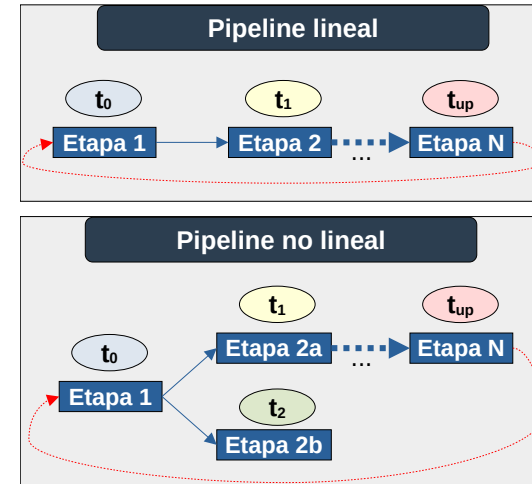
Pipelines

13

- Se deriva de la idea de diseño de paralelismo funcional.
- Un cómputo completo, involucra muchos conjuntos de datos:
 - Flujo de datos a través de varias etapas (**línea de ensamblaje**).
 - Cada etapa una operación diferente sobre los datos.
 - La aplicación se subdivide en subproblemas que se completan secuencialmente dentro de cada etapa.
 - Diferentes etapas se ejecutan en paralelo.
 - Etapas “ociosas”.
 - Máximo paralelismo cuando el pipeline está lleno.
 - *Trade-off* número de etapas vs. comunicación.

Aplicación:

- Paralelismo a nivel de instrucción (ILP)
- Procesamiento gráficos (Pipeline gráfico)
- Procesamiento de señales



**Circular
(Opcional)**



5

Algoritmos sistólicos (Heartbeat)

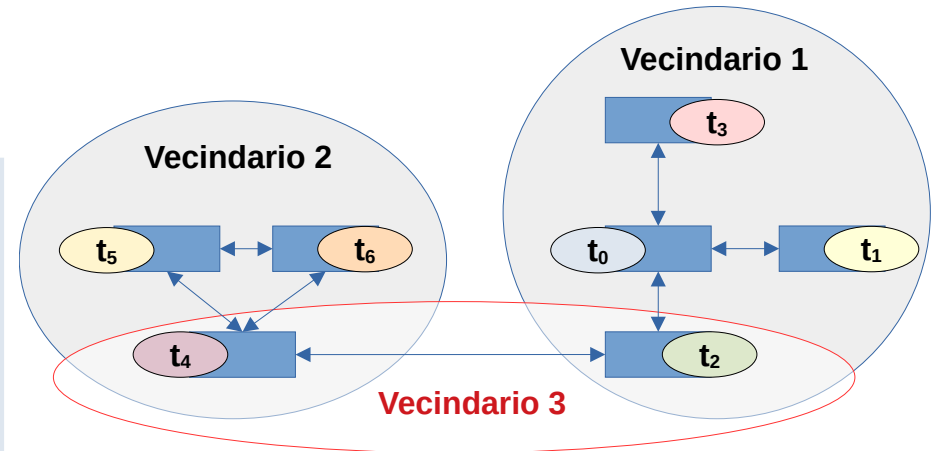
Algoritmos sistólicos (Heartbeat)

15

- Caso particular del paradigma *pipelines*.
- Hilos o procesos cooperan sincrónicamente con vecinos cercanos, intercambiando información en ciclos periódicos de comunicación y cómputo:
 - Secuencia repetitiva (comunicar, procesar y volver a comunicar) análoga al latido del corazón, que en cada pulso impulsa y renueva el flujo a través del sistema.
- Dos etapas repetidas iterativamente hasta alcanzar la solución:
 - Comunicación
 - Cómputo

Aplicación:

- Procesamiento de imágenes
- Solución de ecuaciones diferenciales
- Simulaciones numéricas
- Multiplicación de matrices (Kung y Leiserson)





6

Master-Worker

Master-Worker

17

- Master-Worker (Master-Slave o Task-Farming) se deriva de una de las formas de mapeo dinámico centralizado.

- Dos tipos de hilos o procesos:

Uno o varios Workers

- Solicitan tareas.
- Ejecutan tareas.
- Crean nuevas tareas.
- Envían resultados.

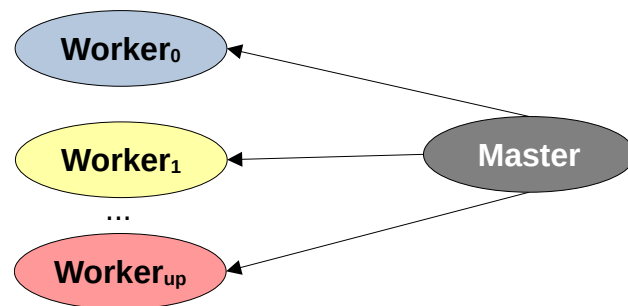
Master

- Controla la ejecución.
- Ejecuta la función principal.
- Puede crear *workers*.
- Crea y distribuye las tareas.
- Recolecta resultados.
- Puede tener rol de *worker*.

- Objetivo: balancear dinámicamente la carga de trabajo.
- Altamente escalables si el número de tareas es mucho mayor que el número de *workers*.
- Sensible al *overhead* por comunicación (cuello de botella).

Aplicación:

- Distribución de cargas de trabajo variables.
- Grandes volúmenes de datos centralizados.
- En distribución estática desbalanceada.





7

Task Pool o Bag of Tasks

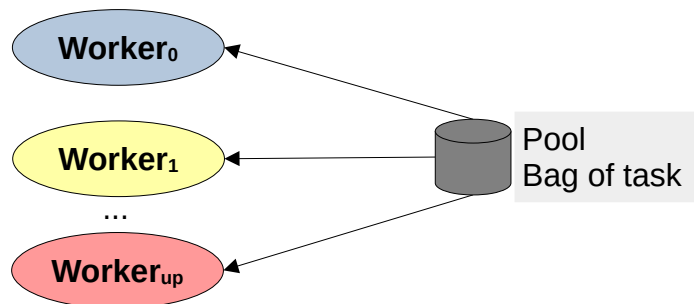
Task pools

19

- *Task Pool* (*Bag of Tasks* o Bolsa de Tareas) se deriva de la otra forma de mapeo dinámico centralizado.
 - Similar al paradigma Master-Worker, pero los workers pueden prescindir de un *Master*.
 - *Workers* acceden directamente a un repositorio centralizado de tareas (**pool o bag of tasks**).
 - Control en el repositorio para evitar condiciones de carrera.
- **Ventaja:** evita el cuello de botella del paradigma *Master-Worker*.

Aplicación:

- Idem Master-Worker.

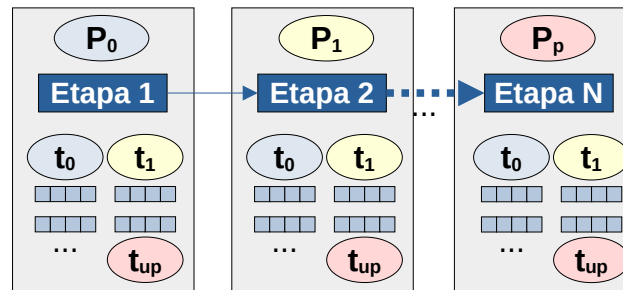




Paradigmas híbridos

21

- Aplicar mas de un paradigma a un problema paralelo.
 - Aprovechar ventajas de diferentes paradigmas y combinarlos según naturaleza del problema.
- Un modelo híbrido puede organizarse:
 - **Jerárquicamente:** Aplicando un paradigma a nivel global y otro dentro de subcomponentes o tareas.
 - **Secuencialmente:** Aplicando distintos paradigmas a diferentes etapas de un algoritmo paralelo.
- **Ejemplo (sobre modelos de arquitectura híbrida):** datos fluyen a través de un pipeline, dentro de cada etapa se aplica paralelismo de datos.



Próxima clase

Introducción a la programación en GPUs

Introducción al concepto de GPGPU

Modelo de programación Nvidia CUDA

